2-merkle_batch_generator

User Instructions

Overview

`2-merkle_batch_generator` is used to collect previously generated manufacturing jobs and organize them into a cryptographically linked batch.

The program scans the structured manufacturing job folders created by:

`1-merkle_integrated_slicer.py`

It then: - reads the generated job manifests - computes leaf hashes - builds a Merkle tree - creates a batch JSON file - optionally prepares the batch for public anchoring

This program is part of the manufacturing accountability workflow.

Launching the Program

Run the script from a terminal:

```
python 2-merkle_batch_generator.py
```

A Tkinter graphical interface window will appear.

Interface Sections

The interface contains several sections used to configure and generate manufacturing batches.

Structured Jobs Root

This is one of the most important sections.

Use the:

`Browse`

button to select the:

`jobs`

folder created by the slicer program.

Example:

```
output/ForgeWorks/Printer01/jobs/
```

This folder contains: - dated manufacturing job folders - G-code files - manifest JSON files - append-only print logs

The batch generator scans this location for valid manufacturing jobs.

Include Submitted

This checkbox determines whether previously submitted jobs should be included again in a new batch.

Unchecked (recommended)

Only jobs that have NOT previously been batched are included.

This is the normal operating mode.

Checked

Previously submitted jobs may be included again.

This is mainly useful for: - testing - rebuilding batches - demonstrations - recovery workflows

---

Dry Run

When enabled, the program performs a simulated batch generation.

It: - scans jobs - computes hashes - builds the Merkle tree

But does NOT permanently write a final batch file.

This is useful for: - testing - demonstrations - validating folder structures

---

Description Field

This optional text field allows you to include notes about the generated batch.

Examples:

```
Weekly Manufacturing Batch
```

```
Prototype Print Demonstration

Public Demo Submission
```

The description becomes part of the batch metadata.

Build Batch Button

Press:

```
Build Batch
```

to generate the manufacturing batch.

When pressed, the program:

1. Scans the selected jobs folder
2. Finds valid manifest files
3. Computes cryptographic leaf hashes
4. Builds a Merkle tree
5. Generates a Merkle root
6. Creates a batch summary
7. Writes a batch JSON file
8. Updates local batch tracking information

Generated Batch Files

The generated batch file is typically named similar to:

```
batch_<facility_id>_<instrument_id>_<unique_id>.json
```

Example:

```
batch_ForgeWorks_Printer01_9f2a1c4d.json
```

The batch file contains: - batch metadata - Merkle root - job references - leaf hashes - proofs - batch hashes - optional descriptions

This file is later used for: - public anchoring - audit verification - remote accountability checks

---

Result Window

After batch generation completes, the:

```
Result
```

window displays a summary of the operation.

This section typically includes: - number of jobs discovered - number of jobs included - Merkle root - generated batch ID - batch file location - warnings or skipped jobs - batch creation status

The Result window is intended as a human-readable summary of what the batch generator produced.

Batch Storage

Generated batch files are typically written into the manufacturing output structure.

Example:

```
output/ForgeWorks/Printer01/
```

or a nearby batch folder depending on configuration.

The batch JSON file can later be: - uploaded to the audit portal - publicly anchored - included in audit bundles - remotely verified

---

Recommended Workflow

Typical usage:

1. Generate manufacturing jobs with:

   ```
   1-merkle_integrated_slicer.py
   ```

2. Open:

   ```
   2-merkle_batch_generator.py
   ```

3. Select the:

   ```
   jobs
   ```

   folder created by the slicer

4. Optionally enter a description

5. Press:

   ```
   Build Batch
   ```

6. Review the Result window

7. Locate the generated batch JSON file

Purpose of the Demonstration

This program demonstrates:

- append-only manufacturing accountability
- Merkle tree batching
- cryptographic manufacturing identity
- structured manufacturing history
- traceable audit preparation

The generated batches can later be: - publicly anchored - compared during audits - remotely verified - used to demonstrate manufacturing consistency